

Begonnen am	Donnerstag, 23. Januar 2025, 22:38
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 12:23
Verbrauchte Zeit	13 Stunden 44 Minuten

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

Frage 1

Richtig

Nicht bewertet

Linkage in C

Markieren Sie, wie die Symbole im Quelltext vom Linker aufgelöst werden.

```
// main.c
#include "square.h"
static uint32_t length;
uint8_t something;

int main(void) {
    uint32_t res;

    ...
    res = calculate_weight(length);
    ...
}

// square.c
...
uint32_t calculate_x(uint32_t number) {
    uint8_t i = 0;
    uint32_t r;

    ...

    return r;
}
```

internal linkage
external linkage
external linkage
no linkage

external linkage
internal linkage
no linkage

external linkage
no linkage
no linkage

Die Antwort ist richtig.

`static uint32_t length;`

Statische globale Variablen sind nur innerhalb des jeweiligen Moduls zu sehen und werden damit nicht nach aussen sichtbar gemacht. Internal Linkage.

`uint8_t something;`

Global und nicht statisch --> External linkage.

`main()`

Global und nicht statisch --> External linkage.

`uint32_t res;`

`res` ist eine private Variable vom `main()` und ist ausserhalb davon nicht sichtbar --> no linkage.

`calculate_x()` --> wie `main()`

`i` und `r` --> wie `res`.

Frage 2

Richtig

Nicht bewertet

Linkage in C

Füllen Sie die folgende Tabelle aus:

Art	Verfügbarkeit	Beispiel
External linkage	Der globale Name kann in beliebigen Modulen verwendet werden	eine Funktion oder eine globale Variable
Internal linkage	Der globale Name ist nur für die Verwendung in diesem Modul verfügbar	eine Funktion oder eine globale Variable
No linkage	Jeder Name, der sich nicht im globalen Raum befindet	lokale Variablen

Die Antwort ist richtig.

Die richtige Antwort lautet:

Linkage in C

Füllen Sie die folgende Tabelle aus:

Art	Verfügbarkeit	Beispiel
External linkage	[Der globale Name kann in beliebigen Modulen verwendet werden]	[eine Funktion oder eine globale Variable]
Internal linkage	[Der globale Name ist nur für die Verwendung in diesem Modul verfügbar]	[eine Funktion oder eine globale Variable]
No linkage	[Jeder Name, der sich nicht im globalen Raum befindet]	[lokale Variablen]

Frage 3

Richtig

Nicht bewertet

Eigenschaften des .lst-Files

Von den folgenden Aussagen über .lst-Files stimmen genau 3.

Welche?

- ☐ 1. Die im .lst-File gezeigten Codierungen sind nicht komplett, weil der Assembler noch nichts über den Ziel-Prozessor (Target CPU) weiss.
- ☒ 2. Das .lst-File wird nach dem Compilieren vom Assembler erzeugt.
- ☐ 3. Die Adressen, die im .lst-File gezeigt werden, entsprechen denen, die man im Debugger antrifft.
- ☐ 4. Für jedes Include-File (.h Datei) wird beim Übersetzen auch ein .lst-File erzeugt.
- ☒ 5. Aus dem .lst-File kann man bereits exakt lesen, wieviel Platz die AREAs eines Moduls benötigen werden.
- ☒ 6. Im .lst-File können einige Instruktionen noch nicht fertig assembliert sein, weil dem Assembler gewisse Informationen fehlen.
- ☐ 7. Sämtliche .lst-Files werden zwischen den Schritten 'Präprozessor' und 'Compiler' erzeugt.

Die Antwort ist richtig.

Die richtigen Antworten sind: Aus dem .lst-File kann man bereits exakt lesen, wieviel Platz die AREAs eines Moduls benötigen werden., Das .lst-File wird nach dem Compilieren vom Assembler erzeugt., Im .lst-File können einige Instruktionen noch nicht fertig assembliert sein, weil dem Assembler gewisse Informationen fehlen.

Frage 4

Richtig

Nicht bewertet

Eigenschaften des .map-Files

Von den folgenden Aussagen über .map-Files stimmen genau 3. Welche?

- ☒ 1. Die Bezeichnung 'map' (=Karte) bezieht sich darauf, dass hier dokumentiert ist, welches Symbol sich wo im Speicher befindet.
- ☒ 2. Im .map-File sind die Adressen aller Objekte wie AREAs, Variablen und Funktionen so gelistet, wie sie auch im Debugger erscheinen.
- ☐ 3. Das .map-File enthält Angaben darüber, wie Instruktionen codiert werden sowie etwaige Include-Fehler.
- ☐ 4. Für jedes Object-File (Modul) wird beim Übersetzen auch ein .map-File erzeugt.
- ☐ 5. Mittels der Angaben im .map-File berechnet der Assembler, wieviel Platz jede AREA benötigt.
- ☐ 6. Das .map-File wird zwischen den Schritten 'Präprozessor' und 'Compiler' erzeugt.
- ☒ 7. Das .map-File enthält Informationen über den Linking-Prozess.

Die Antwort ist richtig.

Die richtigen Antworten sind: Die Bezeichnung 'map' (=Karte) bezieht sich darauf, dass hier dokumentiert ist, welches Symbol sich wo im Speicher befindet.,

Das .map-File enthält Informationen über den Linking-Prozess., Im .map-File sind die Adressen aller Objekte wie AREAs, Variablen und Funktionen so gelistet, wie sie auch im Debugger erscheinen.

◀ [Lab Modular Coding and Linking](#)

Direkt zu:

[Lecture Exceptional Control Flow – Interrupts](#) ▶



Datenschutz | ✉ Supportanfrage